

Plan de projet expérimental — IFT-7026

Université Laval

Étudiants	Léopold Chappuis, Alexandre Eberhardt
Superviseur	Prof. Mohamed Aymen Saied
Directeur de programme	Prof. Pascal Tesson
Session	Session d'automne 2026

Sujet : Un ordonnanceur Linux appris par sélection de candidats : de l'émulateur au noyau réel via `sched_ext`

1. Objectif du projet

Ce projet cherche à savoir si une politique d'ordonnancement *apprise* peut faire mieux que simplement imiter les ordonnanceurs classiques du noyau Linux (CFS, EEVDF), tout en restant assez légère pour être **déployée dans un noyau réel**. Le critère principal est l'amélioration du temps de réponse, *sous contraintes* : sans famine et sans dégradation notable des autres métriques (équité, commutations de contexte, turnaround) — optimiser le temps de réponse seul serait trivial au prix de la famine des tâches longues. Le projet prolonge des résultats préliminaires obtenus dans le cadre du cours GLO-7030, où une formulation *candidate-based* évaluée en boucle fermée a déjà montré une réduction d'environ 10,5% du temps de réponse moyen face à CFS dans un émulateur.

Les objectifs spécifiques sont :

- Améliorer la qualité du signal d'apprentissage : aujourd'hui le modèle apprend à imiter un *teacher* fait de règles fixes. Il sera remplacé par un **teacher quasi-optimal** qui, pour chaque décision, essaie plusieurs choix dans l'émulateur et regarde quelques pas en avant pour identifier le meilleur. Le modèle apprend ensuite à reproduire ces bonnes décisions sans avoir accès au futur (il n'utilise que l'information disponible au moment de décider).
- Corriger le décalage entre entraînement et utilisation (*covariate shift*) : le modèle est entraîné sur des situations produites par CFS, mais une fois aux commandes il crée ses propres situations, jamais vues à l'entraînement. Le remède (**réentraînement itératif de type DAgger**) consiste à laisser le modèle ordonnancer, à faire étiqueter par le *teacher* les situations ainsi créées, puis à réentraîner sur ces nouvelles données.
- Caractériser le compromis **précision / latence d'inférence / nombre de paramètres** par distillation (front de Pareto), la latence étant le principal facteur limitant côté systèmes.
- **Déployer** le modèle dans un vrai noyau Linux, pas seulement dans l'émulateur. Le mécanisme `sched_ext` (Linux ≥ 6.12) permet de brancher un ordonnanceur personnalisé sans recompiler le noyau : sur le modèle de `scx_rustland`, un petit composant dans le noyau transmet chaque décision à un programme en espace utilisateur, qui interrogera ici le modèle distillé pour choisir la prochaine tâche. On mesurera alors, sur de vraies charges (p. ex. compilation, serveur sous requêtes), si la machine répond mieux qu'avec l'ordonnanceur par défaut (EEVDF), avec les mêmes métriques que dans l'émulateur.

2. Résumé

L'ordonnanceur du noyau Linux prend des milliers de décisions par seconde, qui déterminent latence, équité et utilisation du CPU. Ces décisions reposent sur des heuristiques affinées sur des décennies (CFS, puis EEVDF depuis Linux 6.6), qui n'optimisent pas explicitement des objectifs comme le temps de réponse. La question posée est de savoir si un modèle appris peut

reconstruire, voire améliorer, ces décisions à partir de l'état d'exécution. L'application d'un modèle appris à la décision d'ordonnancement, et plus encore son déploiement dans un noyau réel, reste peu explorée dans la littérature : les travaux existants (dont *KernelOracle* [3]) [1–4] se limitent à la prédiction hors ligne, sans déploiement ni évaluation en boucle fermée dans un noyau réel.

Le projet s'appuie sur une base existante (GLO-7030) comportant deux parties : (i) un pipeline d'imitation séquentielle (modèle récurrent LSTM) sur traces `sched_switch` réelles collectées avec `perf`, conservé ici comme **baseline** (« imitation pure », proche de *KernelOracle*); et (ii) une formulation *candidate-based* apprenant à choisir une tâche parmi des candidats à partir d'un état d'ordonnancement riche, via un encodeur à attention sur l'ensemble des tâches et l'historique de décisions, évaluée en boucle fermée dans un émulateur. C'est cette seconde partie qui constitue la contribution principale.

Le projet proposé vise à corriger les limites de ces travaux préliminaires : un *teacher* heuristique trop simple, un *covariate shift* non traité, et une évaluation restreinte à l'émulateur. Il propose un *teacher* quasi-optimal, un réentraînement DAGger, une analyse du compromis précision/latence par distillation, puis un **déploiement dans un noyau réel via `sched_ext`**. L'accent est mis autant sur la **méthodologie d'évaluation** (protocole en boucle fermée, baselines EEVDF/CFS, métriques dont la latence de queue p99, tests statistiques multi-graines) et la **reproductibilité** du pipeline que sur l'apprentissage lui-même : il ne s'agit pas seulement d'entraîner un modèle, mais de mesurer précisément et de façon reproductible ce qu'il apporte. Les entraînements s'appuieront sur les ressources de calcul (serveur ICE, via le compte de recherche du superviseur); le déploiement `sched_ext` ciblera une machine disposant d'un noyau Linux 6.12 (`CONFIG_SCHED_CLASS_EXT`) avec accès privilégié. Le résultat final visé est un rapport au format article, soumissible à un atelier de type *ML for Systems* (repli : EuroMLSys, APSys).

3. Contenu et travaux à effectuer

Le projet est structuré en trois phases.

Phase 0 — Consolidation et protocole expérimental

- Nettoyage et reproductibilité du code existant (pipeline, émulateur).
- Unification du *baseline* de comparaison sur EEVDF (cohérence avec le noyau réel; correction de l'écart CFS/EEVDF des travaux préliminaires).
- Définition d'une suite de charges de travail, des métriques (temps de réponse moyen et **latence de queue p99**, turnaround, commutations de contexte, équité) et d'un protocole statistique (multi-graines, significativité).
- Revue de littérature et positionnement.

Phase 1 — Contribution méthodologique (émulateur)

- *Teacher* quasi-optimal à lookahead et distillation vers un modèle causal.
- Réentraînement itératif DAGger sur les trajectoires auto-induites.
- Front de Pareto précision / latence d'inférence / nombre de paramètres.
- Évaluation en boucle fermée vs EEVDF et CFS, ablations de l'architecture *candidate-based*, généralisation à des charges non vues.

Phase 2 — Déploiement noyau réel

- Intégration d'un ordonnanceur `sched_ext` en espace utilisateur (type `scx_rustland`) interrogeant le modèle distillé.
- Mesures en boucle fermée sur charges réelles vs EEVDF; analyse du budget de latence d'inférence.

- *Repli* : si le déploiement `sched_ext` s'avère bloquant, bascule vers un ordonnanceur « orientable » (politique unique conditionnée par un vecteur de poids d'objectifs), évalué en émulateur.

Liste détaillée des travaux :

1. Effectuer la revue de littérature (imitation, prédiction séquentielle, RL/imitation pour l'ordonnancement, `sched_ext`) et positionner la contribution.
2. Consolider le code et fixer le protocole expérimental (charges, métriques dont p99, statistiques) ; aligner le *baseline* sur EEVDF.
3. Concevoir le *teacher* quasi-optimal à lookahead et le comparer au *teacher* heuristique des travaux préliminaires.
4. Entraîner par distillation et tracer le front de Pareto précision / latence / paramètres ; intégrer le réentraînement DAgger.
5. Évaluer en boucle fermée dans l'émulateur (vs EEVDF/CFS), réaliser les ablations et les tests de généralisation ; rédiger le rapport de mi-session.
6. Intégrer la politique distillée dans un ordonnanceur `sched_ext` et mesurer sur un noyau réel face à EEVDF ; analyser le budget de latence.
7. Rédiger le rapport final au format article et identifier les pistes de recherche futures.

Faisabilité et portée. La fondation déjà en place (code, émulateur et premiers résultats issus de GLO-7030) réduit le risque et concentre l'effort sur la contribution. Les phases 0 et 1, réalisables dans l'enveloppe de 5–6 h/semaine par personne sur la session, constituent le **livrable garanti** du projet. Le déploiement dans un noyau réel (phase 2) en est l'**objectif ambitieux** : en cas de blocage technique sur `sched_ext`, le repli « orientable » décrit ci-dessus assure une contribution expérimentale complète et évaluable en émulateur.

4. Modalités d'évaluation

Composante	Pondération
Présentation orale — état de l'art et positionnement	15 %
Production expérimentale et reproductibilité (Phase 1 : code, émulateur, <i>benchmark</i>)	15 %
Rapport de mi-session (présenté à la présentation intermédiaire)	35 %
Description de la problématique et de la méthodologie	10 %
Analyse quantitative (boucle fermée, ablations, Pareto)	15 %
Analyse qualitative (analyse d'erreurs, limites)	10 %
Rapport final (format article, soutenu à la présentation finale)	35 %
Nouveaux aspects étudiés depuis la mi-session	10 %
Analyse quantitative (<code>sched_ext</code> vs EEVDF, latence)	15 %
Analyse qualitative	10 %
Total	100 %

Les présentations intermédiaire et finale ne sont pas pondérées séparément : elles constituent la soutenance orale des rapports correspondants et sont évaluées au sein de ceux-ci. La présentation orale de l'état de l'art (début de session) est la seule présentation pondérée de façon autonome. Les présentations peuvent se tenir **en visioconférence**.

5. Échéancier

Étape	Date cible
Début du projet (Phase 0)	Début septembre 2026
Présentation orale — état de l’art et positionnement	Fin septembre 2026
Livrable 1 — résultats Phase 1 + rapport de mi-session	Fin octobre 2026
Présentation intermédiaire (soutenance du rapport de mi-session)	Mi-novembre 2026
Livrable 2 — déploiement <code>sched_ext</code> (Phase 2)	Début décembre 2026
Rapport final (format article) + présentation finale	Mi-décembre 2026

6. Livrables

- Code reproductible : pipeline d’apprentissage, émulateur, intégration `sched_ext`.
- Suite de charges de travail / *benchmark* et jeux de décisions candidates utilisés pour les expérimentations.
- Rapport de mi-session et rapport final.
- Présentations : présentation orale (état de l’art), présentation intermédiaire et présentation finale.
- Draft d’article soumissible (cible : *ML for Systems*; repli : EuroMLSys, APSys).

7. Références (préliminaires)

- [1] Jingde Chen, Subho S. Banerjee, Zbigniew T. Kalbarczyk et Ravishankar K. Iyer. *Machine learning for load balancing in the linux kernel*. Dans *Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems*, APSys ’20, p. 67–74, New York, NY, USA, 2020. Association for Computing Machinery.
- [2] Anusha G. H., Minal K., B. P. Varsha, Geethishree Mishra, Meghana G. K. et Madhusudhan K. N. *ML engine for linux scheduler*. *Quest Journals, Journal of Software Engineering and Simulation*, vol. 9, n° 7, p. 20–29, 2023.
- [3] Sampanna Yashwant Kahu. *KernelOracle : Predicting the linux scheduler’s next move with deep learning*, 2025.
- [4] Atul Negi et P. Kishore Kumar. *Applying machine learning techniques to improve linux process scheduling*. Dans *TENCON 2005 – 2005 IEEE Region 10 Conference*, p. 1–6, 2005.
- [5] Stéphane Ross, Geoffrey J. Gordon et Drew Bagnell. *A reduction of imitation learning and structured prediction to no-regret online learning*. Dans *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 15 de *Proceedings of Machine Learning Research*, p. 627–635, 2011.
- [6] Vasuki Shankar. *Machine learning for linux kernel optimization : Current trends and future directions*. *International Journal of Computer Sciences and Engineering*, vol. 13, p. 56–64, 03 2025.
- [7] Ion Stoica et Hussein Abdel-Wahab. *Earliest Eligible Virtual Deadline First : A flexible and accurate mechanism for proportional share resource allocation*. Rapport technique TR-95-22, Old Dominion University, 1995.
- [8] *Sched_ext : BPF extensible scheduler class*. Documentation du noyau Linux. <https://docs.kernel.org/scheduler/sched-ext.html> (consulté en 2026).